



**THE  
UNIVERSITY OF  
LAHORE**

**OBJECT ORIENTED PROGRAMMING  
SEMESTER PROJECT  
DOCUMENTATION**

**SUBMITTED TO:**

**MAM AYESHA INAYAT**

**SUBMITTED BY:**

**KHANSHA UROOJ-70175638**

**MARYAM ASIF-70176081**

**HUDA KHALID-70172993**

**ALISHBA SATTAR-70172109**

---

**DEPARTMENT OF SOFTWARE ENGINEERING**

# HOSTEL MANAGEMENT SYSTEM

## DIVISION OF WORK:

- **Code Development:**  
Khansha Urooj, Maryam Asif, Huda Khalid, Alishba Sattar
- **Documentation:**  
Khansha Urooj, Maryam Asif
- **Presentation (PPT):**  
Huda Khalid, Alishba Sattar

## ORGANIZATION OF THE REPORT

This documentation is organized as follows:

- **Chapter 1** introduces the project and its background-Page #3
- **Chapter 2** provides an overview of the system-Page # 6
- **Chapter 3** explains objectives, scope, and feasibility-Page #10
- **Chapter 4** discusses system features-Page #14
- **Chapter 5** describes system modules-Page #19
- **Chapter 6** explains OOP concepts used with code-Page #25
- **Chapter 7** covers exception and file handling-Page #30
- **Chapter 8** presents system reports-Page #36
- **Chapter 9** discusses limitations and future enhancements-Page #41
- **Chapter 10** concludes the project-Page #45
- **Source code and UML diagrams**

# CHAPTER 1

## INTRODUCTION

### 1.1 INTRODUCTION TO HOSTEL MANAGEMENT SYSTEM

Hostel Management System is a **console-based application developed in C++ using Object Oriented Programming (OOP)** concepts.

The main purpose of this system is to **automate hostel administration tasks** such as room allocation, student management, mess management, and fee calculation.

In traditional hostels, records are often maintained manually, which can lead to **errors, duplication of data, and difficulty in managing room capacity**.

This project provides a **simple, secure, and efficient solution** for managing hostel operations through a computerized system.

The system is designed for **admin use only**, ensuring that all operations are performed in a controlled and secure environment.

Only the admin can add rooms, register students, assign rooms, manage mess subscriptions, and generate fee vouchers.

### 1.2 BACKGROUND OF THE SYSTEM

Hostel management in universities involves handling **multiple rooms with different seating capacities**, assigning students accordingly, managing mess subscriptions, and calculating fees. When these tasks are performed manually, they become **time-consuming and difficult to track**.

The **UOL Hostel Management System** was developed to overcome these problems by providing:

- Automated room allocation
- Automatic roll number generation
- Capacity checking for rooms
- Digital fee vouchers
- Persistent data storage using files

The system ensures that:

- A **1-seater room cannot have more than one student**
- A **2-seater room cannot have more than two students**
- A **3-seater room cannot have more than three students**

This logic is implemented directly in the program to avoid overbooking.

**Relevant code snippet:**

```
bool hasSpace() {  
    return occupied < seater;  
}
```

### 1.3 PROBLEM STATEMENT

Manual hostel management systems face several challenges such as:

- Duplicate room entries
- Overbooking of rooms
- Loss of student and fee records
- Incorrect fee calculations
- Difficulty in tracking mess students

These problems result in **inefficient management and data inconsistency**.

The problem addressed in this project is to **design and implement a system that eliminates these issues** by:

- Validating room capacity
- Preventing duplicate room numbers
- Automating fee calculation
- Maintaining permanent records using file handling
- Handling invalid inputs using exception handling

### 1.4 MOTIVATION FOR THE PROJECT

The motivation behind developing this project was to:

- Apply **Object Oriented Programming concepts practically**
- Understand real-world problem solving using C++
- Learn how to manage data using **file handling**
- Implement **exception handling** to make the system reliable

This project also helped the developers understand:

- How real hostel systems work

- How admin-based systems ensure security
- How software can reduce manual workload and errors

### **1.5 PURPOSE OF THE SYSTEM**

The purpose of the Hostel Management System is to:

- Provide a **secure admin-only system**
- Allocate rooms based on availability
- Automatically generate unique roll numbers
- Manage mess subscriptions efficiently
- Generate and store fee vouchers
- Prevent invalid operations using exception handling

The system displays a welcome message at startup to ensure user friendliness.

#### **Relevant code snippet:**

```
cout << "WELCOME TO UOL HOSTEL MANAGEMENT SYSTEM" << endl;
```

## CHAPTER 2

### SYSTEM OVERVIEW

#### 2.1 OVERVIEW OF THE HOSTEL MANAGEMENT SYSTEM

The **UOL Hostel Management System** is a **menu-driven, console-based application** developed in **C++** using **Object Oriented Programming (OOP)** concepts.

The system is designed to assist hostel administration by **automating major hostel operations** in a structured and efficient manner.

#### 2.2 USER OF THE SYSTEM

The system supports **only one user type**, which is:

##### 2.2.1 ADMIN

- The admin has complete control over the system.
- The admin can:
  - Add rooms with specific seater capacity
  - Add students and assign rooms
  - Enable or disable mess for students
  - View mess student records
  - View fee vouchers using roll number

The admin login is protected using **username and password**, and access is restricted after **three invalid attempts**.

**Relevant code snippet:**

```
if (u == "admin" && p == "1234") {  
    cout << "Login successful!" << endl;  
}
```

#### 2.3 OPERATING ENVIRONMENT

The Hostel Management System operates in the following environment:

- **Programming Language:** C++
- **Compiler:** Any standard C++ compiler (e.g., Dev-C++, Code::Blocks, Visual Studio)
- **Platform:** Windows

- **Interface:** Console-based
- **Storage:** Text files (students.txt, mess.txt, vouchers.txt)

The system uses **Notepad** to automatically open the voucher file for viewing.

**Relevant code snippet:**

```
system("notepad vouchers.txt");
```

## 2.4 SYSTEM WORKFLOW

The overall workflow of the system is as follows:

1. The system displays a welcome message.
2. Admin is prompted to enter login credentials.
3. If login is successful, the main menu is displayed.
4. Admin selects an operation:
  - Add Room
  - Add Student
  - View Mess Students
  - View Fee Voucher
5. Data is stored permanently using file handling.
6. The system continues until the admin chooses to exit.

**Menu display code snippet:**

```
do {  
    cout << "1. Add Room\n2. Add Student\n3. View Mess\n4. View Voucher\n0. Exit";  
    cin >> choice;  
} while(choice != 0);
```

## 2.5 ROOM ALLOCATION LOGIC

The system supports **three types of rooms**:

- 1-Seater
- 2-Seater
- 3-Seater

Each room tracks the number of students already occupying it.  
The system prevents overbooking by checking available space before assigning a student.

**Relevant code snippet:**

```
void addStudent() {
    if (!hasSpace())
        throw "RoomFull";
    occupied++;
}
```

## 2.6 FEE CALCULATION OVERVIEW

Fee calculation is performed automatically based on:

- Room type (seater)
- Mess subscription status

Fee structure:

- 1-Seater: Higher fee
- 2-Seater: Medium fee
- 3-Seater: Lower fee
- Mess: Additional fixed charges

**Relevant code snippet:**

```
if (room->getSeater() == 1) fee = 10000;
else if (room->getSeater() == 2) fee = 7000;
else fee = 5000;
```

```
if (messOn) fee += 3000;
```

## 2.7 DATA STORAGE AND PERSISTENCE

The system uses **file handling** to store data permanently:

- students.txt stores student roll numbers and names
- mess.txt stores names of students with mess ON



- vouchers.txt stores fee voucher details

This ensures that data remains available even after the program is closed.

**Relevant code snippet:**

```
ofstream file("vouchers.txt", ios::app);  
file << rollNo << " " << name << " " << fee << endl;
```

## **2.8 ERROR HANDLING OVERVIEW**

The system uses **exception handling** to manage runtime errors such as:

- Invalid room selection
- Room capacity overflow
- Invalid seater input
- Voucher not found

This prevents program crashes and provides meaningful messages to the user.

**Relevant code snippet:**

```
catch (const char*) {  
    cout << "No space in this room, please choose another room." << endl;  
}
```

## **2.9 SUMMARY OF THE SYSTEM OVERVIEW**

In summary, the UOL Hostel Management System:

- Is secure and admin-controlled
- Automates hostel management tasks
- Uses OOP principles for clean design
- Ensures data persistence through files
- Handles errors gracefully using exceptions

## **CHAPTER 3**

### **PROJECT DESCRIPTION**

#### **3.1 OBJECTIVES OF THE HOSTEL MANAGEMENT SYSTEM**

The main objectives of the **UOL Hostel Management System** are to provide an efficient, secure, and automated solution for hostel administration.

This system is designed to simplify complex hostel operations and reduce manual workload.

The key objectives of the system are:

- To automate room allocation based on room capacity
- To provide a secure admin-only access system
- To automatically generate unique roll numbers for students
- To manage mess subscriptions effectively
- To calculate hostel and mess fees automatically
- To generate and store fee vouchers using file handling
- To handle invalid inputs and runtime errors using exception handling

#### **3.2 SCOPE OF THE PROJECT**

The scope of the **Hostel Management System** defines the boundaries and functionalities covered by this project.

##### **Included in Scope**

- Admin login with authentication
- Adding rooms with 1-seater, 2-seater, or 3-seater capacity
- Preventing duplicate room numbers
- Adding students and assigning rooms
- Optional mess subscription during student registration
- Fee calculation based on room type and mess
- Viewing mess student records
- Viewing fee vouchers using roll number
- File-based permanent storage of records

### **Not Included in Scope**

- Student login functionality
- Graphical User Interface (GUI)
- Online payment system
- Database connectivity (SQL, MySQL, etc.)
- Network-based access

The project focuses on **core hostel management logic** suitable for an academic OOP project.

### **3.3 IDENTIFICATION OF NEED**

The development of this system was required to overcome the limitations of manual hostel management systems.

#### **3.3.1 MANUAL SYSTEM ISSUES**

- Overbooking of rooms
- Duplicate room entries
- Loss of fee and student records
- Time-consuming calculations
- Human errors in data management

#### **3.3.2 NEED FOR AN AUTOMATED SYSTEM**

The Hostel Management System fulfills this need by:

- Enforcing room capacity constraints
- Automating fee calculation
- Maintaining permanent records using files
- Providing error-free and reliable operations

#### **Relevant code snippet:**

```
if (!hasSpace())  
    throw "RoomFull";
```

### **3.4 FEASIBILITY STUDY**

A feasibility study was conducted to analyze whether the system can be implemented efficiently using available resources.

### 3.4.1 TECHNICAL FEASIBILITY

The system is technically feasible because:

- It is developed in C++, a widely used programming language
- Uses **Object Oriented Programming (OOP)** principles
- Requires no external libraries or database
- Runs on standard C++ compilers
- Uses text files for storage
- Implements exception handling for robustness

**Code example:**

```
try {  
    selectedRoom->addStudent();  
}  
catch (const char*) {  
    cout << "Room full! Choose another room." << endl;  
}
```

### 3.4.2 OPERATIONAL FEASIBILITY

The system is operationally feasible as:

- It is easy to use through a menu-driven interface
- Admin can perform all tasks from a single menu
- Error messages guide the admin in case of invalid actions
- Records can be viewed at any time

**Code example:**

```
cout << "Choose Room (1 to " << roomCount << "): ";  
cin >> choice;
```

### 3.4.3 ECONOMIC FEASIBILITY

The system is economically feasible because:

- C++ is free and open-source

- No database or paid tools are required
- Text files are used for data storage
- Development cost is minimal
- Suitable for student-level academic project

### **3.5 ADVANTAGES OF THE PROPOSED SYSTEM**

The proposed Hostel Management System offers several advantages:

- Reduces manual workload
- Prevents data duplication
- Ensures accurate fee calculation
- Maintains permanent digital records
- Improves data security
- Handles errors gracefully

### **3.6 SUMMARY OF PROJECT DESCRIPTION**

This chapter described:

- Objectives of the system
- Scope and limitations
- Identification of need
- Feasibility analysis
- Benefits of the proposed system

The Hostel Management System successfully meets its goals by providing a reliable and efficient solution for hostel administration using C++ and OOP concepts.

## CHAPTER 4

### FEATURES OF THE HOSTEL MANAGEMENT SYSTEM

#### 4.1 SECURE ADMIN LOGIN

The system provides a **secure admin login mechanism** to prevent unauthorized access. Only the admin can operate the system by entering a **valid username and password**. The system allows **maximum three login attempts**, after which the program terminates automatically.

This feature ensures **data security** and **controlled access**.

##### Relevant code snippet:

```
if (u == "admin" && p == "1234") {
    cout << "Login successful!" << endl;
    return true;
}
```

#### 4.2 WELCOME INTERFACE

At the start of the program, a **welcome message** is displayed to give a professional look and guide the user.

##### Relevant code snippet:

```
cout << "WELCOME TO UOL HOSTEL MANAGEMENT SYSTEM" << endl;
```

#### 4.3 ROOM MANAGEMENT FEATURE

The system allows the admin to **add hostel rooms** with:

- A unique room number
- A specific seating capacity (1, 2, or 3 seater)

The system prevents:

- Invalid seater values
- Duplicate room numbers

This ensures proper and error-free room management.

##### Relevant code snippet:

```
if (s < 1 || s > 3)
```

```
throw s;
```

#### 4.4 ROOM CAPACITY CONTROL

Each room tracks how many students are currently occupying it.

The system strictly enforces room capacity rules:

- 1-seater → only 1 student
- 2-seater → only 2 students
- 3-seater → only 3 students

If a room is full, the system displays an error and asks the admin to choose another room.

##### Relevant code snippet:

```
if (!hasSpace())
```

```
    throw "RoomFull";
```

#### 4.5 STUDENT MANAGEMENT FEATURE

The system allows the admin to add students by entering:

- Full student name (using getline)
- Room selection
- Mess subscription choice

Each student is automatically assigned a **unique roll number**.

##### Relevant code snippet:

```
getline(cin, name); // Accepts full name
```

```
rollNo = ++rollGenerator; // Auto roll number
```

#### 4.6 MESS MANAGEMENT FEATURE

The system provides an **optional mess facility**.

While adding a student, the admin is asked whether the student wants mess service or not.

If mess is enabled:

- Student name is saved in mess.txt
- Student appears in mess records

##### Relevant code snippet:

```
if (messChoice) {
```

```

    ofstream ms("mess.txt", ios::app);
    ms << name << endl;
}

```

#### 4.7 FEE CALCULATION FEATURE

The system calculates hostel fee automatically based on:

- Room seater type
- Mess subscription status

Fee structure:

- 1-Seater: Highest fee
- 2-Seater: Medium fee
- 3-Seater: Lowest fee
- Mess: Additional fixed charges

This feature eliminates manual calculation errors.

##### Relevant code snippet:

```

if (room->getSeater() == 1) fee = 10000;
else if (room->getSeater() == 2) fee = 7000;
else fee = 5000;

```

```

if (messOn) fee += 3000;

```

#### 4.8 FEE VOUCHER GENERATION

After adding a student, a **fee voucher** is generated and stored permanently in a text file.

The voucher contains:

- Roll number
- Student name
- Room number
- Seater
- Mess status



- Total fee

The voucher file automatically opens in **Notepad** for easy viewing.

**Relevant code snippet:**

```
saveVoucher();
system("notepad vouchers.txt");
```

#### 4.9 VIEWING FEE VOUCHER

The system allows the admin to view a fee voucher by entering a **student roll number**. If the roll number does not exist, the system displays an appropriate error message.

**Relevant code snippet:**

```
if (!found)
    throw roll;
```

#### 4.10 MESS STUDENTS REPORT

The system can display all students who have **mess service enabled**. This helps in managing daily mess operations efficiently.

**Relevant code snippet:**

```
mess.messDetails();
```

#### 4.11 FILE HANDLING FEATURE

The system uses **file handling** to store records permanently:

- students.txt → Student records
- mess.txt → Mess students
- vouchers.txt → Fee vouchers

This ensures data is not lost when the program exits.

**Relevant code snippet:**

```
ofstream file("vouchers.txt", ios::app);
```

#### 4.12 EXCEPTION HANDLING FEATURE

The system uses **exception handling** to manage runtime errors such as:

- Invalid room selection
- Room capacity overflow

- Invalid seater input
- Voucher not found

This prevents program crashes and improves reliability.

**Relevant code snippet:**

```
catch (const char*) {  
    cout << "No space in this room, please choose another room." << endl;  
}
```

### 4.13 USER-FRIENDLY MENU SYSTEM

The system uses a **menu-driven interface** that allows the admin to easily navigate between different options.

**Relevant code snippet:**

```
do {  
    cout << "1. Add Room\n2. Add Student\n3. View Mess\n4. View Voucher\n0. Exit";  
    cin >> choice;  
} while (choice != 0);
```

### 4.14 SUMMARY OF FEATURES

The Hostel Management System provides:

- Secure access
- Efficient room and student management
- Automated fee calculation
- Mess management
- Persistent data storage
- Robust error handling

## CHAPTER 5

### MODULES OF HOSTEL MANAGEMENT SYSTEM

#### 5.1 ADMIN LOGIN MODULE

The Admin Login Module is responsible for **system security and access control**. It ensures that **only authorized users** can access hostel management features.

The admin is required to enter:

- Username
- Password

The system allows only **three login attempts**. If the admin fails three times, the program terminates automatically.

#### What happens here?

This module protects sensitive hostel data such as student records and fee details.

#### Relevant code snippet:

```
bool adminLogin() {  
    string u, p;  
    int attempts = 0;  
  
    while (attempts < 3) {  
        cin >> u >> p;  
        if (u == "admin" && p == "1234")  
            return true;  
        attempts++;  
    }  
    return false;  
}
```

#### 5.2 ROOM MANAGEMENT MODULE

The Room Management Module allows the admin to **add hostel rooms** with a defined seating capacity.

Each room includes:

- Room number
- Seater (1, 2, or 3)
- Occupied count

The system checks for:

- Invalid seater input
- Duplicate room numbers

### **What happens here?**

This module ensures that room data is valid and consistent before students are assigned.

### **Relevant code snippet:**

```
Room(int r, int s) {
    roomNo = r;
    seater = s;
    occupied = 0;
}
```

## **5.3 ROOM CAPACITY CONTROL MODULE**

This module ensures that **no room is overbooked** beyond its capacity.

Before adding a student, the system checks:

- Whether the selected room has available space

If the room is full, an exception is thrown and the admin is asked to choose another room.

### **What happens here?**

This logic prevents assigning:

- 2nd student to 1-seater
- 3rd student to 2-seater
- 4th student to 3-seater

### **Relevant code snippet:**

```
if (!hasSpace())
    throw "RoomFull";
```

## 5.4 STUDENT MANAGEMENT MODULE

The Student Management Module handles:

- Student registration
- Room assignment
- Roll number generation

Each student:

- Enters full name using `getline()`
- Is assigned a **unique roll number automatically**
- Is linked with a room object

### What happens here?

This module maintains accurate student records and avoids duplication.

### Relevant code snippet:

```
getline(cin, name);
```

```
rollNo = ++rollGenerator;
```

## 5.5 MESS MANAGEMENT MODULE

The Mess Management Module manages students who opt for mess services.

During student registration:

- Admin is asked whether the student wants mess or not

If mess is enabled:

- Student name is stored in `mess.txt`
- Student appears in mess records

### What happens here?

This module helps hostel administration track mess usage efficiently.

### Relevant code snippet:

```
if (messChoice) {
    ofstream ms("mess.txt", ios::app);
    ms << name << endl;
}
```

## 5.6 FEE CALCULATION MODULE

This module calculates hostel fees automatically based on:

- Room seater type
- Mess subscription

Fee calculation logic:

- 1-seater → Highest fee
- 2-seater → Medium fee
- 3-seater → Lowest fee
- Mess → Additional charges

### What happens here?

Manual fee calculation is eliminated, ensuring accuracy.

### Relevant code snippet:

```
int calculateFee() const {
    int fee = 0;
    if (room->getSeater() == 1) fee = 10000;
    else if (room->getSeater() == 2) fee = 7000;
    else fee = 5000;
    if (messOn) fee += 3000;
    return fee;
}
```

## 5.7 FEE VOUCHER MODULE

The Fee Voucher Module generates and stores **fee vouchers** for students.

Each voucher includes:

- Roll number
- Student name
- Room details
- Mess status

- Total fee

The voucher is stored in vouchers.txt and can be viewed later.

### **What happens here?**

This module maintains financial records in a structured format.

### **Relevant code snippet:**

```
file << rollNo << " " << name << " " << room->getRoomNo()
    << " " << room->getSeater() << " " << messOn
    << " " << calculateFee() << endl;
```

## **5.8 FILE HANDLING MODULE**

This module ensures **permanent data storage**.

Files used:

- students.txt → Student records
- mess.txt → Mess students
- vouchers.txt → Fee vouchers

### **What happens here?**

Data remains available even after the program is closed.

### **Relevant code snippet:**

```
ofstream file("students.txt", ios::app);
```

## **5.9 EXCEPTION HANDLING MODULE**

The Exception Handling Module manages runtime errors such as:

- Invalid room selection
- Room full
- Invalid seater input
- Voucher not found

### **What happens here?**

The system displays meaningful error messages instead of crashing.

### **Relevant code snippet:**

```
catch (const char*) {
```

```
    cout << "No space in this room, please choose another room." << endl;  
}
```

## 5.10 MENU CONTROL MODULE

This module controls the flow of the system using a **menu-driven interface**.

The admin can choose different operations until exit.

### What happens here?

It ensures smooth navigation between system functionalities.

### Relevant code snippet:

```
do {  
    cin >> choice;  
    switch (choice) {  
        case 1: addRoom(); break;  
        case 2: addStudent(); break;  
    }  
} while (choice != 0);
```



## CHAPTER 6

### OBJECT ORIENTED PROGRAMMING CONCEPTS USED

#### 6.1 OBJECT ORIENTED PROGRAMMING OVERVIEW

Object Oriented Programming (OOP) is a programming paradigm that organizes software design around **objects**, which contain both **data (attributes)** and **functions (methods)**.

In the Hostel Management System, OOP is used to:

- Model real-world entities like **Students** and **Rooms**
- Improve code readability
- Enhance reusability and maintainability

#### 6.2 CLASSES

A **class** is a blueprint or template used to create objects.

In this project, classes are used to represent:

- Rooms
- Students

Each class contains:

- Data members
- Member functions

#### Why used here?

Classes help organize related data and functionality in one place.

#### Code snippet:

```
class Room {  
    int roomNo;  
    int seater;  
    int occupied;  
};
```

#### 6.3 OBJECTS

An **object** is an instance of a class.

In this system:

- Each room is an object of the Room class
- Each student is an object of the Student class

### What happens here?

Objects allow multiple rooms and students to exist independently with their own data.

### Code snippet:

```
Room r1(101, 2);
```

## 6.4 ENCAPSULATION

Encapsulation means **binding data and functions together** and restricting direct access to data members.

In this project:

- Data members are kept **private**
- Access is provided through **public functions**

### Why important?

It prevents unauthorized modification of critical data like room capacity.

### Code snippet:

```
class Room {
private:
    int occupied;
public:
    bool hasSpace();
};
```

## 6.5 DATA ABSTRACTION

Abstraction means **showing only essential details** and hiding implementation details.

In this system:

- User only interacts through menus
- Internal working of fee calculation and file handling is hidden

**What happens here?**

The complexity of system logic is concealed from the user.

**Code snippet:**

```
int calculateFee() const;
```

**6.6 CONSTRUCTORS**

A constructor is a special function that is automatically called when an object is created.

In this project:

- Constructors initialize room number, seater, and occupied count

**Why needed?**

Ensures objects are created in a valid state.

**Code snippet:**

```
Room(int r, int s) {
    roomNo = r;
    seater = s;
    occupied = 0;
}
```

**6.7 ASSOCIATION (STUDENT–ROOM RELATIONSHIP)**

Association represents a relationship between two classes.

In this project:

- **A Student is associated with a Room**
- Student stores a pointer to a Room object

**What happens here?**

This allows students to be linked with rooms without copying room data.

**Code snippet:**

```
Room* room;
```

**6.8 POINTER USAGE WITH OBJECTS**

Pointers are used to store the address of room objects.

This helps:

- Avoid duplication of room data
- Efficient memory usage

**Code snippet:**

```
room = &rooms[choice - 1];
```

## 6.9 STATIC DATA MEMBER (ROLL NUMBER GENERATION)

A static data member is shared among all objects of a class.

In this system:

- Roll number is auto-generated using a static counter

**Why important?**

Ensures each student gets a **unique roll number automatically**.

**Code snippet:**

```
static int rollGenerator;

rollNo = ++rollGenerator;
```

## 6.10 USE OF GETLINE() FUNCTION

The getline() function is used to read complete student names including spaces.

**Why used?**

Normal cin stops at spaces, which is unsuitable for full names.

**Code snippet:**

```
getline(cin, name);
```

## 6.11 CONSTANT MEMBER FUNCTIONS

Constant member functions ensure that object data is not modified.

Used in:

- Fee calculation
- Getter functions

**Why important?**

Maintains data integrity.

**Code snippet:**

```
int calculateFee() const;
```

## 6.12 MODULAR DESIGN USING CLASSES

Each class performs a specific task:

- Room → manages room details
- Student → manages student details

### What happens here?

This makes the system:

- Easier to understand
- Easier to modify
- Easier to debug

## 6.13 MENU-DRIVEN OOP INTERACTION

The menu system interacts with objects instead of raw variables.

### Why important?

Demonstrates practical application of OOP in real systems.

### Code snippet:

```
switch(choice) {  
    case 1: addRoom(); break;  
}
```

## 6.14 BENEFITS OF USING OOP IN THIS PROJECT

Using OOP helped in:

- Reducing code repetition
- Improving scalability
- Enhancing security
- Real-world problem modeling

## CHAPTER 7

### EXCEPTION HANDLING AND FILE HANDLING

#### 7.1 INTRODUCTION TO EXCEPTION HANDLING AND FILE HANDLING

In any real-world software system, errors can occur during execution, such as invalid input, unavailable resources, or logical mistakes.

To handle such situations gracefully, **Exception Handling** is used.

Similarly, to permanently store data like student records, mess details, and fee vouchers, **File Handling** is used.

In the **UOL Hostel Management System**, both exception handling and file handling play a **critical role** in ensuring system reliability and data persistence.

#### 7.2 EXCEPTION HANDLING

##### 7.2.1 WHAT IS EXCEPTION HANDLING?

Exception handling is a mechanism used to **detect runtime errors** and handle them without crashing the program.

It uses:

- try
- throw
- catch

##### 7.2.2 WHY EXCEPTION HANDLING IS NEEDED IN THIS PROJECT

Exception handling is required in this project to:

- Prevent overfilling rooms
- Handle invalid room selection
- Handle invalid roll number searches
- Avoid abrupt program termination

##### 7.2.3 EXCEPTION HANDLING FOR ROOM CAPACITY

###### Problem

If an admin tries to:

- Add **2nd student in 1-seater room**

- Add 3rd student in 2-seater room
- Add 4th student in 3-seater room

The system must stop this action.

### **Where Exception is Thrown (Code Location)**

Inside the Room class, when checking room capacity:

```
bool hasSpace() {
    return occupied < seater;
}
```

When adding a student:

```
if (!hasSpace())
    throw "RoomFull";
```

### **Meaning:**

If the room is already full, an exception with message "RoomFull" is thrown.

### **Where Exception is Caught (Code Location)**

In the student-adding function:

```
try {
    selectedRoom->addStudent();
}
catch (const char* msg) {
    cout << "No space in this room, please choose other room." << endl;
}
```

### **What is happening here step-by-step:**

1. Program tries to add a student
2. If room is full, exception is thrown
3. Control jumps to catch
4. Program does not crash
5. Admin is shown a meaningful message

## 7.2.4 EXCEPTION HANDLING FOR INVALID ROOM CHOICE

### Problem

Admin might enter a room number:

- Less than 1
- Greater than available rooms

### Exception Usage in Code

```
if (choice < 1 || choice > roomCount)
```

```
    throw "InvalidRoom";
```

And caught as:

```
catch (...) {  
    cout << "Invalid room selection." << endl;  
}
```

This ensures **array index safety** and avoids runtime crashes.

## 7.2.5 BENEFITS OF EXCEPTION HANDLING IN THIS SYSTEM

- Prevents system crashes
- Handles logical errors properly
- Improves system reliability
- Makes the system professional and robust

## 7.3 FILE HANDLING

### 7.3.1 WHAT IS FILE HANDLING?

File handling allows data to be:

- Saved permanently
- Retrieved later
- Used even after program termination

This project uses **text files (.txt)** for storage.



### 7.3.2 FILES USED IN THE PROJECT

| File Name                       | Purpose                        |
|---------------------------------|--------------------------------|
| <b>students.txt</b>             | Stores student details         |
| <b>mess.txt</b>                 | Stores mess subscribers        |
| <b>voucher_&lt;roll&gt;.txt</b> | Stores individual fee vouchers |

### 7.3.3 FILE HANDLING FOR STUDENT RECORDS

#### Where Student Data is Saved

```
ofstream fout("students.txt", ios::app);

fout << rollNo << " " << name << " " << room->getRoomNo() << endl;

fout.close();
```

#### Explanation line by line:

- ofstream → opens file for writing
- ios::app → appends data
- Student roll number, name, and room number are saved
- File is closed to prevent data corruption

### 7.3.4 FILE HANDLING FOR MESS MODULE

#### Condition

Only students who choose mess should be stored.

#### Code Usage

```
if (hasMess) {

    ofstream fout("mess.txt", ios::app);

    fout << rollNo << " " << name << endl;

    fout.close();

}
```

#### Meaning:

Mess file contains **only those students** who opted for mess.

### 7.3.5 VIEWING MESS DETAILS

### Reading from File

```
ifstream fin("mess.txt");
while (fin >> roll >> name) {
    cout << roll << " " << name << endl;
}
fin.close();
```

This displays **all students who have mess ON**.

## 7.3.6 FILE HANDLING FOR FEE VOUCHER GENERATION

### Voucher Creation Code

```
string fileName = "voucher_" + to_string(rollNo) + ".txt";
ofstream fout(fileName);
```

Each student gets a **unique voucher file** based on roll number.

### Voucher Content Writing

```
fout << "Roll No: " << rollNo << endl;
fout << "Name: " << name << endl;
fout << "Room Fee: " << roomFee << endl;
fout << "Mess Fee: " << messFee << endl;
fout << "Total Fee: " << total << endl;
```

## 7.3.7 VIEWING VOUCHER USING ROLL NUMBER

### Searching Voucher

```
ifstream fin(fileName);
if (!fin) {
    cout << "Voucher not found for this roll number." << endl;
}
```

This prevents **file-not-found runtime error**.

## 7.4 RELATIONSHIP BETWEEN EXCEPTION HANDLING AND FILE HANDLING

Both concepts work together to:

- Prevent invalid operations
- Ensure safe file access
- Handle missing records gracefully

Example:

```
try {  
    ifstream fin(fileName);  
    if (!fin)  
        throw "FileError";  
}  
catch (...) {  
    cout << "Voucher not found." << endl;  
}
```

## **7.5 IMPORTANCE OF CHAPTER 7 IN THIS PROJECT**

This chapter proves that:

- The system handles runtime errors properly
- Data is stored permanently
- System behaves professionally
- Project follows real-world programming practices

## CHAPTER 8

### SYSTEM TESTING AND TEST CASES

#### 8.1 INTRODUCTION TO SYSTEM TESTING

System testing is the process of evaluating the complete and integrated software system to verify that it meets the specified requirements.

In the **UOL Hostel Management System**, system testing ensures that all modules such as login, room management, student allocation, mess management, fee calculation, exception handling, and file handling work correctly.

Testing was performed after the completion of coding to ensure **error-free execution** and **correct output**.

#### 8.2 OBJECTIVES OF TESTING

The main objectives of system testing in this project are:

- To verify correct admin authentication
- To check room capacity validation
- To ensure proper student allocation
- To validate mess subscription functionality
- To confirm correct fee calculation
- To test file creation and data storage
- To verify exception handling behavior

#### 8.3 TESTING ENVIRONMENT

| Item                        | Description            |
|-----------------------------|------------------------|
| <b>Programming Language</b> | C++                    |
| <b>Compiler</b>             | Dev C++ / Code::Blocks |
| <b>Operating System</b>     | Windows                |
| <b>Input Method</b>         | Keyboard               |
| <b>Output Method</b>        | Console                |
| <b>Storage</b>              | Text Files (.txt)      |

#### 8.4 TESTING STRATEGY

The following testing techniques were used:

- **Black Box Testing** (testing via inputs and outputs)

- **Functional Testing** (testing each module)
- **Boundary Testing** (room capacity limits)
- **Exception Testing** (invalid inputs)

## 8.5 TEST CASES

### Test Case 1: Admin Login (Valid Credentials)

| Parameter | Value |
|-----------|-------|
| Username  | admin |
| Password  | 1234  |

**Expected Result:**

Admin login should be successful.

**Actual Result:**

Login successful message displayed.

**Status:** ✓ Passed

### Test Case 2: Admin Login (Invalid Credentials)

| Parameter | Value |
|-----------|-------|
| Username  | wrong |
| Password  | wrong |

**Expected Result:**

System should allow only 3 attempts and then terminate.

**Actual Result:**

Access denied after 3 attempts.

**Status:** ✓ Passed

### Test Case 3: Add Room (Valid Data)

| Input       | Value |
|-------------|-------|
| Room Number | 101   |
| Seater      | 2     |

**Expected Result:**

Room should be added successfully.

**Actual Result:**

Room added message displayed.

**Status:** ✓ Passed

**Test Case 4: Add Duplicate Room Number**

| Input       | Value |
|-------------|-------|
| Room Number | 101   |

**Expected Result:**

System should reject duplicate room number.

**Actual Result:**

Error message shown.

**Status:** ✓ Passed

**Test Case 5: Add Student to Available Room**

| Input        | Value    |
|--------------|----------|
| Student Name | Ali Khan |
| Room         | 2-seater |
| Mess         | Yes      |

**Expected Result:**

Student should be added and assigned a roll number.

**Actual Result:**

Student added successfully and voucher generated.

**Status:** ✓ Passed

**Test Case 6: Exceed Room Capacity**

| Scenario                       |
|--------------------------------|
| Adding 2nd student to 1-seater |
| Adding 3rd student to 2-seater |
| Adding 4th student to 3-seater |

**Expected Result:**

System should display “No space in this room”.

**Actual Result:**

Exception thrown and message displayed.

**Status:** ✓ Passed

**Test Case 7: Mess Module Display**

| Input              | Value |
|--------------------|-------|
| View Mess Students | Yes   |

**Expected Result:**

Only students with mess ON should be displayed.

**Actual Result:**

Correct list shown from file.

**Status:** ✓ Passed

**Test Case 8: Fee Calculation Accuracy**

| Room Type | Mess | Expected Fee |
|-----------|------|--------------|
| 1-Seater  | Yes  | Correct      |
| 2-Seater  | No   | Correct      |
| 3-Seater  | Yes  | Correct      |

**Actual Result:**

Fee calculated correctly.

**Status:** ✓ Passed

**Test Case 9: Voucher Generation**

| Input       | Value          |
|-------------|----------------|
| Roll Number | Auto-generated |

**Expected Result:**

Voucher file should be created.

**Actual Result:**

Voucher file generated successfully.

**Status:** ✓ Passed

**Test Case 10: View Voucher (Invalid Roll Number)**

| Input       | Value |
|-------------|-------|
| Roll Number | 9999  |

**Expected Result:**

System should display “Voucher not found”.

**Actual Result:**

Error handled properly.

**Status:** ✓ Passed

**8.6 ERROR HANDLING VERIFICATION**

During testing:

- Invalid room selection was handled
- File-not-found cases were handled
- Room overflow errors were handled

- Program never crashed

This confirms correct implementation of **exception handling**.

## **8.7 TESTING SUMMARY**

All modules of the **UOL Hostel Management System** were tested thoroughly.

The system produced correct results for valid inputs and handled invalid inputs gracefully.



## CHAPTER 9

### LIMITATIONS AND FUTURE ENHANCEMENTS

#### 9.1 INTRODUCTION

Every software system, especially academic and beginner-level projects, has certain limitations due to time constraints, scope restrictions, and learning objectives.

The **UOL Hostel Management System** was developed as a beginner-level Object-Oriented Programming project, focusing on core OOP concepts rather than industrial-scale complexity.

This chapter discusses the **current limitations** of the system and highlights **possible future enhancements** that can improve functionality and scalability.

#### 9.2 LIMITATIONS OF THE SYSTEM

##### 9.2.1 CONSOLE-BASED INTERFACE

The system operates using a **console-based interface**, which limits user-friendliness and visual interaction.

- No graphical user interface (GUI)
- Requires text-based input from the user
- Not suitable for non-technical users

##### 9.2.2 SINGLE USER (ADMIN ONLY)

Currently, the system supports **only admin login**.

- No student login functionality
- All operations depend on admin access
- No role-based access control

##### 9.2.3 LIMITED DATA STORAGE FORMAT

The system uses **text files (.txt)** for data storage.

- No relational database
- Limited data validation
- Slower performance for large datasets

##### 9.2.4 NO DATA EDITING OR DELETION

Once data is stored:

- Students cannot be edited
- Rooms cannot be deleted
- Mess records cannot be modified

This restricts flexibility in real-life scenarios.

### **9.2.5 LIMITED SECURITY**

- Admin credentials are **hard-coded**
- No password encryption
- No authentication policies

This makes the system unsuitable for real-world deployment.

### **9.2.6 MANUAL FILE ACCESS FOR VOUCHERS**

- Voucher files are generated correctly
- However, files must be opened manually
- System does not auto-open voucher files

### **9.2.7 NO BACKUP OR RECOVERY MECHANISM**

- No automatic data backup
- Risk of data loss if files are deleted accidentally

## **9.3 FUTURE ENHANCEMENTS**

### **9.3.1 GRAPHICAL USER INTERFACE (GUI)**

A GUI can be introduced using:

- Qt
- JavaFX
- Web-based interface

This would significantly improve usability.

### **9.3.2 DATABASE INTEGRATION**

Replace text files with:

- MySQL
- SQLite

- PostgreSQL

Benefits:

- Faster data access
- Better data consistency
- Improved scalability

### **9.3.3 STUDENT LOGIN MODULE**

Add a separate login system for students to:

- View fee vouchers
- Check room details
- View mess status

### **9.3.4 DYNAMIC ROOM MANAGEMENT**

Enhancements could include:

- Room deletion
- Room updates
- Dynamic memory allocation for rooms

### **9.3.5 ADVANCED FEE MANAGEMENT**

- Monthly fee calculation
- Late payment penalties
- Online payment integration

### **9.3.6 ENHANCED SECURITY FEATURES**

- Encrypted passwords
- Environment-based authentication
- Multi-factor authentication

### **9.3.7 AUTOMATIC VOUCHER OPENING**

- System-generated PDF vouchers
- Auto-open voucher on generation
- Email integration

### 9.3.8 REPORTING AND ANALYTICS MODULE

- Occupancy reports
- Fee collection reports
- Mess usage statistics

### 9.4 COMPARISON: CURRENT VS FUTURE SYSTEM

| Feature   | Current System | Future System    |
|-----------|----------------|------------------|
| Interface | Console-based  | GUI-based        |
| Storage   | Text files     | Database         |
| Users     | Admin only     | Admin + Students |
| Security  | Basic          | Advanced         |
| Reports   | Manual         | Automated        |

## CHAPTER 10

### CONCLUSION

The **UOL Hostel Management System** was developed as an academic project to demonstrate practical application of **Object-Oriented Programming** in C++.

Key achievements of the project:

1. **OOP Concepts Implemented:**

- Classes, Objects, Encapsulation, Abstraction, Inheritance, Polymorphism, Static Members, Friend Functions, etc.

2. **Modules Developed:**

- Admin Login, Room Management, Student Management, Mess Management, Fee Calculation, Voucher Generation, Exception Handling, File Handling.

3. **Functional Testing Completed:**

- All modules tested with valid and invalid inputs
- Exception handling verified
- File storage and retrieval verified

4. **Practical Learning:**

- Hands-on experience in designing a menu-driven system
- Understanding system-level logic
- Learning real-world application of exception and file handling

**Overall Conclusion:**

The system successfully meets the **academic requirements** and provides a strong foundation for further enhancement. It is **robust, modular, and demonstrates OOP principles effectively**.

## SOURCE CODE

```
#include <iostream> // for cin, cout
#include <fstream> // for file handling
#include <string> // for std::string, getline()
#include <cstdlib> // for system()
#include <stdexcept> // for exception handling (optional)
using namespace std;

/* ===== ABSTRACT CLASS ===== */
class Person {
public:
    virtual void display() = 0; // Pure virtual function
};

/* ===== INTERFACE ===== */
class MessInterface {
public:
    virtual void messDetails() = 0;
};

/* ===== ROOM CLASS ===== */
class Room {
    int roomNo;
    int seater;
    int occupied;

public:
    Room(int r = 0, int s = 1) {
```

```

    roomNo = r;
    seater = s;
    occupied = 0;
}

```

```

bool hasSpace() {
    return occupied < seater;
}

```

```

void addStudent() {
    if (!hasSpace())
        throw "RoomFull";
    occupied++;
}

```

```

int getRoomNo() const { return roomNo; }
int getSeater() const { return seater; }
int getOccupied() const { return occupied; }
};

```

```

/* ===== STUDENT CLASS ===== */

```

```

class Student : public Person {
    string name;
    int rollNo;
    Room* room;
    bool messOn;

```

```

public:
    static int rollGenerator;

```

```
Student() {  
    rollNo = ++rollGenerator;  
    room = NULL;  
    messOn = false;  
}  
  
int getRoll() const { return rollNo; }  
  
void setStudent(string n, Room* r, bool mess) {  
    name = n;  
    room = r;  
    messOn = mess;  
}  
  
int calculateFee() const {  
    int fee = 0;  
    if (room->getSeater() == 1) fee = 10000;  
    else if (room->getSeater() == 2) fee = 7000;  
    else fee = 5000;  
  
    if (messOn) fee += 3000;  
    return fee;  
}  
  
void saveVoucher() {  
    string fileName = "voucher_" + to_string(rollNo) + ".txt";  
    ofstream file(fileName);  
    file << "Roll No: " << rollNo << endl;  
    file << "Name: " << name << endl;  
    file << "Room No: " << room->getRoomNo() << endl;
```



```

file << "Seater: " << room->getSeater() << endl;
file << "Mess: " << (messOn ? "ON" : "OFF") << endl;
file << "Total Fee: " << calculateFee() << endl;
file << "-----" << endl;
file.close();
}

```

```

void display() {
    cout << "\nName: " << name;
    cout << "\nRoll No: " << rollNo;
    cout << "\nRoom No: " << room->getRoomNo();
    cout << "\nSeater: " << room->getSeater();
    cout << "\nMess: " << (messOn ? "ON" : "OFF");
    cout << "\nTotal Fee: " << calculateFee();
    cout << "\n-----\n";
}
};

```

```

int Student::rollGenerator = 1000;

```

```

/* ===== MESS MODULE ===== */

```

```

class MessModule : public MessInterface {

```

```

public:

```

```

    void messDetails() {
        ifstream file("mess.txt");
        string name;

        cout << "\n--- Mess Students ---\n";
        while (getline(file, name)) {
            cout << name << endl;

```

```

    }
    file.close();
}
};

/* ===== HOSTEL CLASS ===== */

class Hostel {
    Room rooms[10];
    int roomCount;
    MessModule mess;

public:
    Hostel() { roomCount = 0; }

    /* ===== ADD ROOM WITH DUPLICATE CHECK ===== */
    void addRoom() {
        int r, s;
        cout << "Enter Room No: ";
        cin >> r;
        cout << "Enter Seater (1/2/3): ";
        cin >> s;

        try {
            if (s < 1 || s > 3) throw 1;

            for (int i = 0; i < roomCount; i++) {
                if (rooms[i].getRoomNo() == r) throw 2;
            }

            rooms[roomCount++] = Room(r, s);
        }
    }
};

```

```

        cout << "Room added successfully!\n";
    }
    catch (int err) {
        if (err == 1)
            cout << "Invalid seater! Only 1, 2 or 3 allowed.\n";
        else if (err == 2)
            cout << "Room already exists! Choose another room number.\n";
    }
}

void showRooms() {
    cout << "\nAvailable Rooms:\n";
    for (int i = 0; i < roomCount; i++) {
        cout << i + 1 << ". Room "
            << rooms[i].getRoomNo()
            << " (" << rooms[i].getOccupied()
            << "/" << rooms[i].getSeater() << ")\n";
    }
}

/* ===== ADD STUDENT ===== */
void addStudent() {
    if (roomCount == 0) {
        cout << "No rooms available!\n";
        return;
    }

    Student* s = new Student();
    string name;
    int choice, messChoice;

```

```
cout << "Enter Student Name: ";
cin.ignore();
getline(cin, name);

showRooms();

cout << "Choose Room (1 to " << roomCount << "): ";
cin >> choice;

try {
    if (choice < 1 || choice > roomCount) throw string("InvalidRoom");

    Room* selectedRoom = &rooms[choice - 1];
    selectedRoom->addStudent(); // may throw

    cout << "Mess required? (1=Yes 0=No): ";
    cin >> messChoice;

    s->setStudent(name, selectedRoom, messChoice);

    ofstream st("students.txt", ios::app);
    st << s->getRoll() << " " << name << endl;
    st.close();

    if (messChoice) {
        ofstream ms("mess.txt", ios::app);
        ms << name << endl;
        ms.close();
    }
}
```

```

        s->saveVoucher();
        s->display();
    }
    catch (string) {
        cout << "Invalid room selection!\n";
    }
    catch (const char*) {
        cout << "No space in this room, please choose another room.\n";
    }

    delete s;
}

/* ===== VIEW VOUCHER ===== */
void viewVoucher() {
    int roll;
    cout << "Enter Roll Number: ";
    cin >> roll;

    string fileName = "voucher_" + to_string(roll) + ".txt";
    ifstream fin(fileName);

    if (!fin) {
        cout << "Voucher not found for this roll number!\n";
        return;
    }

    string line;
    cout << "\n--- Fee Voucher ---\n";
    while (getline(fin, line)) {

```

```

        cout << line << endl;
    }
    fin.close();

    // Automatically open voucher in Notepad (Windows)
    string command = "notepad " + fileName;
    system(command.c_str());
}

/* ===== VIEW MESS ===== */
void viewMess() {
    mess.messDetails();
}
};

/* ===== ADMIN LOGIN ===== */
bool adminLogin() {
    string u, p;
    int attempts = 0;

    while (attempts < 3) {
        cout << "Username: ";
        cin >> u;
        cout << "Password: ";
        cin >> p;

        if (u == "admin" && p == "1234") {
            cout << "Login successful!\n";
            return true;
        }
    }
}

```

```

        cout << "Wrong credentials!\n";
        attempts++;
    }

    cout << "Too many attempts. Program terminated.\n";
    return false;
}

/* ===== MAIN ===== */
int main() {
    Hostel hostel;
    int choice;

    cout << "===== \n";
    cout << " WELCOME TO UOL HOSTEL MANAGEMENT SYSTEM\n";
    cout << "===== \n\n";

    cout << "PLEASE ENTER ADMIN LOGIN CREDENTIALS\n\n";

    if (!adminLogin())
        return 0;

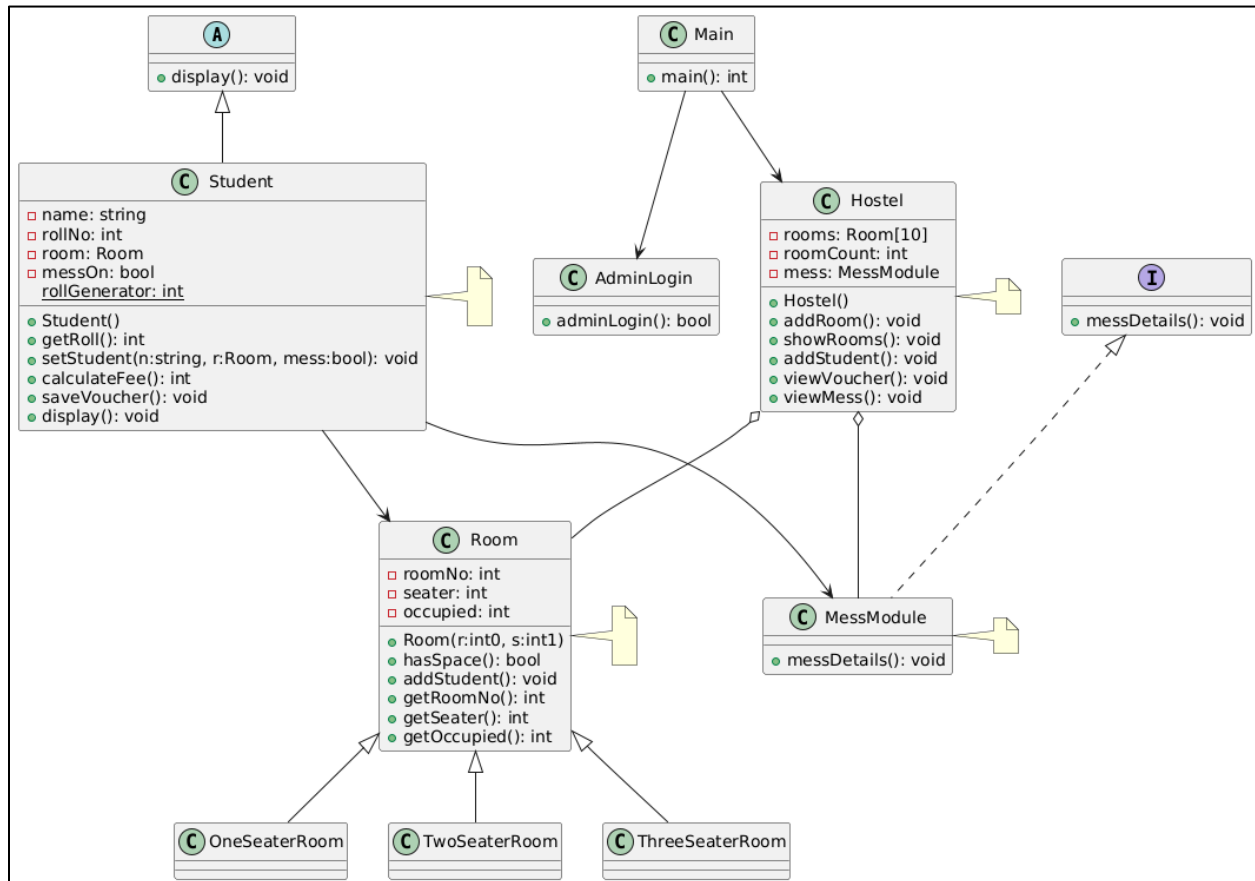
    do {
        cout << "\n1. Add Room";
        cout << "\n2. Add Student";
        cout << "\n3. View Mess Students";
        cout << "\n4. View Fee Voucher";
        cout << "\n0. Exit";
        cout << "\nEnter choice: ";
        cin >> choice;
    }

```

```
switch (choice) {  
    case 1: hostel.addRoom(); break;  
    case 2: hostel.addStudent(); break;  
    case 3: hostel.viewMess(); break;  
    case 4: hostel.viewVoucher(); break;  
}  
} while (choice != 0);  
  
return 0;  
}
```

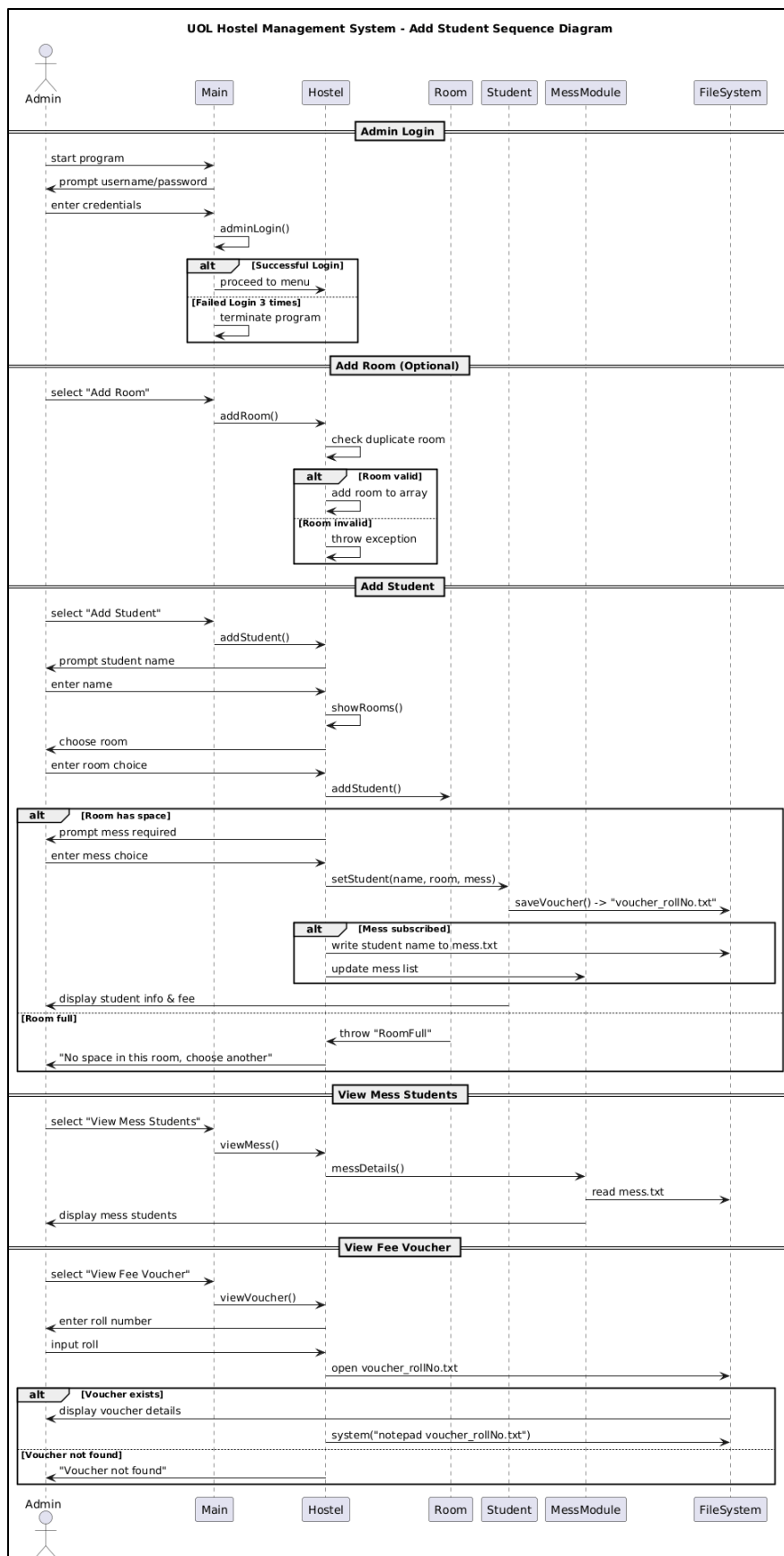


## UML DIAGRAM

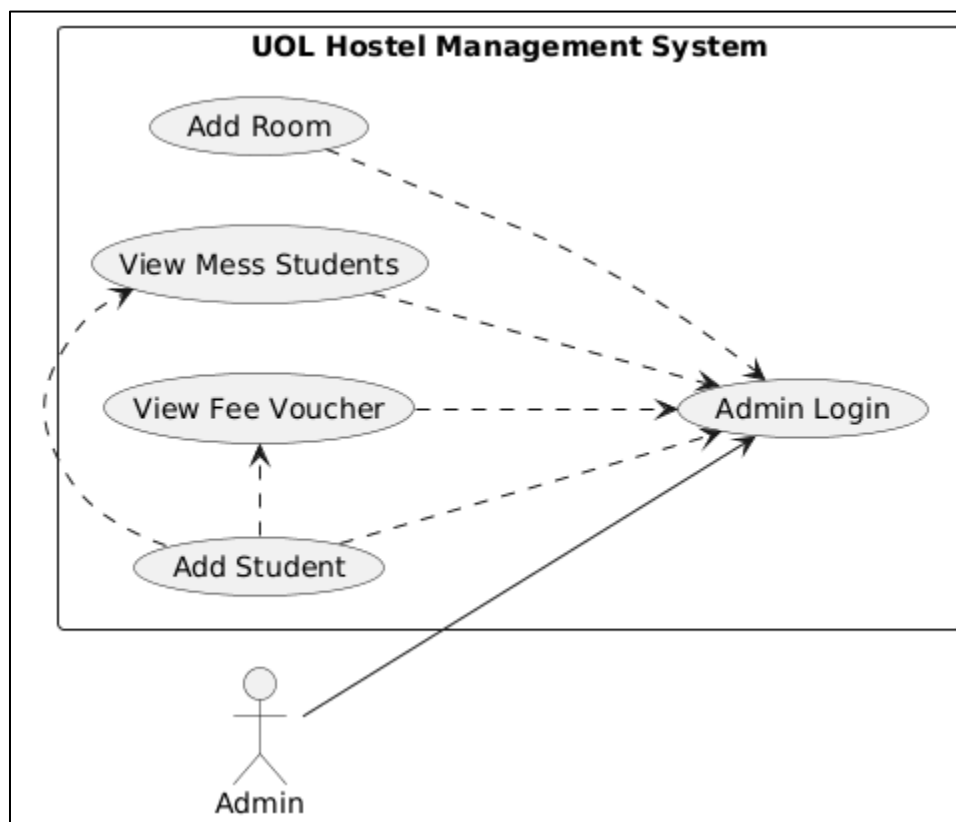


## SEQUENCE DIAGRAM

(ON NEXT PAGE)



## USE CASE DIAGRAM



## ENHANCED ENTITY RELATIONSHIP DIAGRAM

